Number Theory Study Guide

1. Exponentiation

Exponentiation is the mathematical operation of raising a number (base) to a power (exponent). In programming, we often need to calculate a^b where 'a' is the base and 'b' is the exponent.

Basic Exponentiation Implementation

```java
public class Exponentiation {
    // Simple recursive approach
    public static long power(long base, long exponent) {
        if (exponent == 0) {
            return 1;
        }
        return base * power(base, exponent - 1);
    }

    // Iterative approach
    public static long powerIterative(long base, long exponent) {
        long result = 1;
        for (int i = 0; i < exponent; i++) {
            result *= base;
        }
        return result;
    }

    public static void main(String[] args) {
        System.out.println("2^5 = " + power(2, 5));        // Output: 32
        System.out.println("3^4 = " + powerIterative(3, 4)); // Output: 81
    }
}
```

Time Complexity

- Simple recursive/iterative: O(n) where n is the exponent
- Problem: Very slow for large exponents

2. Binary Exponentiation

Binary exponentiation (also called fast exponentiation) is an efficient algorithm to calculate a^b in $O(\log b)$ time instead of $O(b)$. It uses the property that:

- If b is even: $a^b = (a^{(b/2)})^2$
- If b is odd: $a^b = a \times a^{(b-1)}$

Binary Exponentiation Implementation

```java
public class BinaryExponentiation {
    // Recursive approach
    public static long fastPower(long base, long exponent) {
        if (exponent == 0) {
            return 1;
        }

        if (exponent % 2 == 0) {
            // Even exponent
            long halfPower = fastPower(base, exponent / 2);
            return halfPower * halfPower;
        } else {
            // Odd exponent
            return base * fastPower(base, exponent - 1);
        }
    }

    // Iterative approach
    public static long fastPowerIterative(long base, long exponent) {
        long result = 1;
        while (exponent > 0) {
            if (exponent % 2 == 1) {
                result *= base;
            }
            base *= base;
            exponent /= 2;
        }
        return result;
    }

    public static void main(String[] args) {
        System.out.println("2^10 = " + fastPower(2, 10));           // Output: 1024
        System.out.println("5^6 = " + fastPowerIterative(5, 6));    // Output: 15625
    }
}
```

Binary Exponentiation with Modulo

When dealing with large numbers, we often need to calculate (a^b) % m to prevent overflow:

```java
public class ModularExponentiation {
    public static long modPower(long base, long exponent, long modulo) {
        long result = 1;
        base = base % modulo;

        while (exponent > 0) {
            if (exponent % 2 == 1) {
                result = (result * base) % modulo;
            }
            exponent = exponent >> 1; // Same as exponent /= 2
            base = (base * base) % modulo;
        }
        return result;
    }

    public static void main(String[] args) {
        // Calculate (2^20) % 1000000007
        System.out.println("(2^20) % 1000000007 = " + modPower(2, 20, 1000000007));
    }
}
```

Time Complexity

- Binary exponentiation: O(log b) where b is the exponent
- Much faster than simple exponentiation for large exponents

3. Inverse Modulo

The modular multiplicative inverse of an integer 'a' modulo 'm' is an integer 'x' such that: **a × x ≡ 1 (mod m)**

The inverse exists only when gcd(a, m) = 1 (a and m are coprime).

Finding Inverse Using Extended Euclidean Algorithm

java

```java
public class InverseModulo {
    // Extended Euclidean Algorithm
    public static int[] extendedGCD(int a, int b) {
        if (b == 0) {
            return new int[]{a, 1, 0}; // {gcd, x, y}
        }

        int[] result = extendedGCD(b, a % b);
        int gcd = result[0];
        int x1 = result[1];
        int y1 = result[2];

        int x = y1;
        int y = x1 - (a / b) * y1;

        return new int[]{gcd, x, y};
    }

    // Find modular inverse of a modulo m
    public static int modInverse(int a, int m) {
        int[] result = extendedGCD(a, m);
        int gcd = result[0];
        int x = result[1];

        if (gcd != 1) {
            throw new ArithmeticException("Modular inverse does not exist");
        }

        // Make x positive
        return (x % m + m) % m;
    }

    public static void main(String[] args) {
        int a = 3, m = 11;
        int inverse = modInverse(a, m);
        System.out.println("Inverse of " + a + " modulo " + m + " is: " + inverse);

        // Verify: (3 * 4) % 11 = 1
        System.out.println("Verification: (" + a + " * " + inverse + ") % " + m + " = "
                           + ((a * inverse) % m));
    }
}
```

Finding Inverse Using Fermat's Little Theorem

When the modulo is a prime number p, we can use Fermat's Little Theorem: **a^(p-1) ≡ 1 (mod p)**
Therefore: **a^(p-2) ≡ a^(-1) (mod p)**

```java
public class FermatInverse {
    // Using binary exponentiation for modular inverse
    public static long modInverseFermat(long a, long p) {
        // p must be prime
        return modPower(a, p - 2, p);
    }

    public static long modPower(long base, long exponent, long modulo) {
        long result = 1;
        base = base % modulo;

        while (exponent > 0) {
            if (exponent % 2 == 1) {
                result = (result * base) % modulo;
            }
            exponent = exponent >> 1;
            base = (base * base) % modulo;
        }
        return result;
    }

    public static void main(String[] args) {
        long a = 3, p = 7; // p is prime
        long inverse = modInverseFermat(a, p);
        System.out.println("Inverse of " + a + " modulo " + p + " is: " + inverse);

        // Verify
        System.out.println("Verification: (" + a + " * " + inverse + ") % " + p + " = "
                           + ((a * inverse) % p));
    }
}
```

Applications of Inverse Modulo

- Modular division: (a/b) % m = (a × b^(-1)) % m
- Solving linear congruences

- Number theory problems
- Cryptography algorithms

Quiz Questions

1. What is the time complexity of binary exponentiation? a) O(n) b) O(log n) c) O(n^2) d) O(1)

Answer: b) O(log n)

2. When does the modular multiplicative inverse of 'a' modulo 'm' exist? a) Always b) When a > m c) When gcd(a, m) = 1 d) When m is prime

Answer: c) When gcd(a, m) = 1

3. What is 2^10 calculated using binary exponentiation? a) 512 b) 1024 c) 2048 d) 256

Answer: b) 1024

4. Which theorem can be used to find modular inverse when the modulo is prime? a) Euler's theorem b) Fermat's Little Theorem c) Chinese Remainder Theorem d) Wilson's theorem

Answer: b) Fermat's Little Theorem

5. In binary exponentiation, what do we do when the exponent is even? a) Multiply base by itself b) Subtract 1 from exponent c) Square the result of base^(exponent/2) d) Add base to result

Answer: c) Square the result of base^(exponent/2)

Practice Coding Questions

1. Fast Power with Modulo Write a Java method that calculates (base^exponent) % modulo efficiently using binary exponentiation. The method should handle large values without overflow.

2. Modular Inverse Calculator Implement a Java program that finds the modular multiplicative inverse of a number 'a' modulo 'm' using the Extended Euclidean Algorithm. Include proper error handling for cases where the inverse doesn't exist.

3. Power of Two Check Write a Java method that determines if a given positive integer n can be expressed as 2^k for some non-negative integer k. Use bit manipulation techniques for an efficient solution.

4. Modular Division Create a Java method that performs modular division (a/b) % m by finding the modular inverse of b and then multiplying it with a. Handle the case when b and m are not coprime.

5. Fast Fibonacci with Matrix Exponentiation Implement a Java solution to find the nth Fibonacci number using matrix exponentiation. Use binary exponentiation to achieve O(log n) time complexity.